Array Divisibility Reconstruction

Here's a concise explanation you can keep in your notes.

Intuition

We are given the final array `b`.

Originally, there was an array `a` such that $a_i \mid a_{i+1}$ (meaning a_i divides a_{i+1} seamlessly).

Then Bob multiplied some elements by the same number x.

Our task is **not** to find the original array `a`. We only need to find one possible valid value for x.

Key Observation

If `b[i+1] % b[i] == 0`, then this adjacent pair is already valid. No information about x is obtained from it.

If `b[i+1] % b[i] != 0`, then the divisibility property is broken. This mismatch can only happen because the first element (`b[i]`) was multiplied by x, disrupting the ratio.

Why compute `b[i] / gcd(b[i], b[i+1])`?

Consider an invalid pair with values **20** and **12**:

> **Numerical Case Study:** Pair (20, 12)
> - `gcd(20, 12) = 4`
> - `20 / 4 = 5`
>
> - $20 = 4 \times 5$
> - $12 = 4 \times 3$
>
> The common part (4) already existed in both numbers. The remaining part (5) exists only in the first element (20). This extra isolated factor must have come from Bob's multiplication.
>
> **Conclusion:** x must contain the factor 5.
>
> *Note:* This does not mean we multiply 12 by 5. It means originally the first element was $20 / 5 = 4$, and Bob multiplied it by 5 to get 20.

Why take LCM?

Each bad pair provides a localized constraint:

- One bad pair may dictate: *"x must contain factor 3."*
- Another bad pair may dictate: *"x must contain factor 5."*

To satisfy all structural constraints simultaneously, x must be a multiple of every needed factor. The smallest number that satisfies all independent criteria is their Least Common Multiple (LCM).

Therefore, running `ans = lcm(ans, need)` sequentially combines all individual requirements into a single valid value for x.

Algorithm

```
ans = 1

for every adjacent pair:
    if b[i+1] % b[i] == 0:
        continue

    g = gcd(b[i], b[i+1])
    need = b[i] / g
    ans = lcm(ans, need)

print ans
```

Doubts & Answers

Q1. Why not divide every element by the GCD of the whole array?

Because the original array configuration is not unique, and dividing globally by the absolute GCD of the entire array does not guarantee the sequential prefix-to-suffix divisibility property is preserved.

Q2. Why use b[i] / gcd?

It specifically isolates and extracts the extra factor present exclusively in `b[i]`, which must have been injected via Bob's multiplication step.

Q3. Why use LCM?

Each bad pair gives a localized requirement: x must be divisible by `need`. To fulfill all conditions across the sequence, x must be divisible by every single requirement. The minimal universal number doing so is the LCM.

Q4. Why ignore good pairs?

Because they already naturally satisfy $b[i] \mid b[i+1]$ without intervention, meaning they place absolutely no new restrictions or constraints on the choice of x.

ONE-LINE SUMMARY

Every bad adjacent pair tells us one factor that x must contain (`b[i] / gcd(b[i], b[i+1])`). Taking the LCM of all these extracted factors gives a valid x that satisfies every single broken pair simultaneously.